Université Clermont Auvergne – ISIMA

Lab 1 – Solving the Jump Game

Problem Solving Algorithms

Erkin Tunc Boya | Jean-Baptiste Largeron

Group 2 • 22 October 2026

Abstract

This report models the *Jump Game* as a search problem and documents our Java implementation in progress. It presents the rules, a backtracking search design, and the roles of the existing classes. Gradle, JUnit 5, and GitHub Actions are used in the project workflow. Features not yet present in the supplied code are identified separately.

Contents

1 Introduction 2

2 Problem Description 2

3 Formalization as a Search Problem 2

4 State-Space Graph 3

5 Backtracking Search 3

6 Java Implementation and Tools 4

7 Tests and Remaining Work 4

8 Conclusion 4

GitHub Repository 4

How to Run 5

1 Introduction

The Jump Game is played on an $n \times m$ board containing colored tokens and empty cells. The aim is to leave exactly one token after a sequence of valid jumps. Our project models this puzzle as a search problem and explores backtracking as a possible solution method.

2 Problem Description

2.1 Rules

A token can jump horizontally or vertically over a token of another color and land on an empty cell. Several consecutive tokens may be removed in one jump if they share the same color, which must differ from the moving token. The jumped tokens are removed from the board. The four directions match our `Direction` enum.

2.2 Example

Here R and G represent different colors and $\varnothing$ an empty cell:

$$[\,R\;G\;G\;\varnothing\,] \quad \longrightarrow \quad [\,\varnothing\;\varnothing\;\varnothing\;R\,].$$

The red token jumps over two green tokens. Only one token remains.

3 Formalization as a Search Problem

3.1 State space

A state is a board configuration of size $n \times m$, with $n, m \geq 1$. In Java, `State` stores it in an `int[][]` array: `0` means empty, and positive integers represent colors. The state space S contains all configurations reachable by legal jumps.

3.2 Initial state

The initial state s_0 is the given board. It contains at least one token and at least one empty cell.

3.3 Goal test

A state is a goal when exactly one token remains. This condition is checked by `State.isFinal()`.

3.4 Dead-End situation

A state is in a dead-end situation if there is more than one colored token and it is impossible to jump then we can backtrack or if all the ways were already explored, there is no solution.

3.5 Actions and legality conditions

An action specifies a starting position, a direction, the moving token's color, and the color and number of jumped tokens. Our `Action` class stores these values. A legal jump must cross one or more consecutive tokens of the same, different color and end at an empty cell inside the board.

3.6 Successor function

The transition $\text{succ}(s, a) = s'$ moves the selected token to the empty destination and removes the jumped tokens. In our intended design, the existing board is updated rather than copied for each move.

3.7 Action cost and path cost

We use a unit cost per legal jump. Thus, the path cost is the number of jumps in a solution; this is different from the execution time of the program.

4 State-Space Graph

Each node represents a board configuration and each directed edge represents a legal jump. Every move removes at least one token, so the number of tokens strictly decreases and the graph has no cycles. Different move sequences may still lead to the same state.

With k_0 tokens initially, a search path has at most $k_0 - 1$ jumps. This limits the recursion depth but does not prevent a large number of branches.

5 Backtracking Search

5.1 Principle

Backtracking tries a legal move, explores the resulting state, and undoes the move if it cannot lead to a solution. The following is the *target search algorithm*, not a claim that the recursive solver is already implemented in the supplied Java files.

Algorithm 1 Planned backtracking search

```
1:  function Solve(s)
2:      if Goal(s) then
3:          return true
4:      end if
5:      for all a ∈ A(s) do
6:          ApplyAndRemember(s, a)
7:          if Solve(s) then
8:              return true
9:          end if
10:         UndoLast(s)
11:     end for
12:     return false
13: end function
```

5.2 Current implementation status

`Director` owns the current `State` and a `MemoryOfActions`. The latter keeps an ordered list of actions. At present, `Director.performAction()` records an action but does not keep the board in memory, this implementation . `Director.backtrack()` attempts to undo the last action, but does not yet restore all removed tokens and uses the moving color when calculating a position. These methods must be completed before they can support the algorithm above. Likewise, `Action.isValid()` currently checks only whether the direction is non-null; full move legality is not yet implemented in the supplied classes.

5.3 Complexity

Let k_0 be the initial token count, b the maximum number of legal moves per state, and $d \leq k_0 - 1$ the search depth. In the worst case, a backtracking tree may contain $O(b^d)$ nodes. A reversible, in-place implementation can avoid storing a full board copy at every step. These are theoretical bounds, not measured results.

6 Java Implementation and Tools

The supplied code is organized into a few small classes:

- `State`: stores the grid, changes a cell with `colorCase()`, and checks the goal with `isFinal()`.
- `Direction`: defines `Up`, `Down`, `Left`, and `Right`.
- `Action`: stores the details of one attempted jump.
- `MemoryOfActions`: stores the history of actions.
- `Director`: manages the current state and action history, including the partially implemented undo operation.

We use **Gradle** to run, build, and test the project. The README lists `./gradlew run`, `./gradlew build`, and `./gradlew test`. The project uses **JUnit 5** for automated tests and **GitHub Actions** for continuous integration. The workflow configuration and test sources were not included in the provided snapshot, so their detailed steps and results are not documented here.

7 Tests and Remaining Work

The following are small test cases to check with JUnit 5. They describe expected behavior, not verified test results.

Table 1: Suggested correctness tests (`.` = empty cell).

Input	Expected result	Purpose
`R G .`	One token remains	Single-token jump
`R G G .`	One token remains	Multiple-token jump
`R R .`	No legal jump	Different-color rule
`R . .`	Already solved	Goal test

Next steps are to implement full move validation, apply and reverse board changes correctly, add the recursive search, and verify these behaviors with JUnit 5. The required input/output interface and performance measurements must also be documented after implementation.

8 Conclusion

Our current Java classes provide the board representation, action data, and action history needed for the Jump Game. The formal search model and backtracking procedure describe the intended solver. Completing legal moves and correct undo behavior is the main next step before evaluating search time and memory use.

GitHub Repository

The source code and project files are available in the following repository:

https://github.com/ErkinTunc/M1-ARP

How to Run

The project uses the Gradle wrapper to build, run, and test the Java application. Open a terminal in the project root directory before executing the following commands.

1. Run the Application

Compile and launch the application using the Gradle `run` task.

Windows:

```
.\gradlew.bat --console=plain -q run
```

Linux/macOS:

```
./gradlew --console=plain -q run
```

2. Build the Project

Build the application and run the automated tests:

```
./gradlew build
```

3. Run the Tests

Run the test task independently:

```
./gradlew test
```

After running the tests, open the HTML report in a web browser:

```
build/reports/tests/test/index.html
```

4. Clean the Project

Remove the generated build files:

```
./gradlew clean
```